

Procedimento Operacional Padrão para Conexão com Bancos de Dados PostgreSQL e MongoDB

Gabriel Santos Campos de Andrade (Universidade Federal de Goiás - UFG)

Rafael Santos Campos de Andrade (Universidade Federal de Goiás - UFG)



Este trabalho apresenta a construção de um Procedimento Operacional Padrão (POP) voltado ao acesso e conexão com bancos de dados relacionais e não relacionais, utilizando PostgreSQL e MongoDB. O estudo aborda conceitos relacionados a sistemas de informação, arquitetura de software, modelagem de bancos de dados, normalização e relacionamentos, além de documentar de maneira detalhada os procedimentos necessários para instalação, configuração e conexão com os bancos de dados propostos. O objetivo é desenvolver autonomia técnica e analítica no acesso direto às fontes de dados, permitindo a reprodução dos procedimentos documentados mesmo por usuários com pouca experiência prévia em bancos de dados

Palavras-chave: Engenharia de Produção, Banco de dados, PostgreSQL, MongoDB, POP.

1. Introdução

A transformação digital e o avanço da Indústria 4.0 vêm promovendo mudanças significativas na forma como as organizações coletam, armazenam e utilizam dados em seus processos produtivos. Nesse cenário, a informação passou a assumir papel estratégico dentro das empresas, contribuindo diretamente para atividades de planejamento, controle e tomada de decisão organizacional (Laudon; Laudon, 2014). Dessa forma, a Engenharia de Produção tornou-se cada vez mais dependente de dados confiáveis e atualizados para o gerenciamento eficiente de operações relacionadas à produção, logística, estoque, qualidade e desempenho operacional.

Os Sistemas de Informação Gerencial (SIG) possuem papel fundamental nesse contexto, uma vez que permitem transformar dados brutos em informações úteis para apoio à gestão organizacional (O'Brien; Marakas, 2013). Entretanto, em muitas organizações, os profissionais ainda dependem de relatórios prontos, planilhas exportadas e dashboards previamente configurados, o que pode limitar a flexibilidade das análises realizadas. Além disso, essas abordagens frequentemente apresentam problemas relacionados ao atraso na atualização das informações, excesso de agregação dos dados e dependência do setor de Tecnologia da Informação para execução de novas consultas e análises.

Diante disso, o acesso direto aos bancos de dados torna-se uma importante ferramenta de autonomia analítica para o engenheiro de produção, permitindo consultas mais específicas, maior rastreabilidade das informações e análises alinhadas às necessidades operacionais da organização. Segundo Elmasri e Navathe (2018), os bancos de dados são essenciais para garantir integridade, consistência e organização das informações dentro dos sistemas computacionais modernos, possibilitando maior confiabilidade no armazenamento e recuperação dos dados.

Além disso, o crescimento do volume e da variedade de dados industriais impulsionou o desenvolvimento de diferentes modelos de bancos de dados. Os bancos relacionais, como PostgreSQL e MySQL, utilizam tabelas estruturadas e relacionamentos definidos, sendo amplamente aplicados em sistemas corporativos devido à sua consistência transacional e integridade referencial (Date, 2004). Por outro lado, os bancos de dados não relacionais, também conhecidos como NoSQL, como o MongoDB, surgiram como alternativa para aplicações que demandam maior flexibilidade, escalabilidade e manipulação de dados semiestruturados ou não estruturados (Sadalage; Fowler, 2013).

Nesse contexto, compreender os fundamentos relacionados aos bancos de dados, à arquitetura de software e aos sistemas de informação tornou-se uma competência relevante para profissionais da Engenharia de Produção, especialmente em ambientes industriais cada vez mais orientados por dados. Assim, o presente trabalho tem como objetivo elaborar um Procedimento Operacional Padrão (POP), no formato de artigo científico, para documentar o processo de acesso, conexão, extração e análise de dados em bancos de dados aplicados à Engenharia de Produção. Para isso, serão abordados conceitos relacionados aos Sistemas de Informação Gerencial, modelagem de dados, normalização, relacionamentos, agregações e composições, além da descrição dos procedimentos de conexão utilizando bancos de dados relacionais e não relacionais.

2. Fundamentação teórica

2.1. Sistemas de informação gerencial (SIG)

Os Sistemas de Informação Gerencial (SIG) podem ser definidos como sistemas responsáveis pela coleta, processamento, armazenamento e distribuição de informações utilizadas no apoio à gestão organizacional e à tomada de decisão (Laudon; Laudon, 2014). Em ambientes empresariais cada vez mais competitivos e orientados por dados, os SIG tornaram-se ferramentas essenciais para integração das informações provenientes dos diversos setores das organizações, permitindo maior controle, agilidade e eficiência operacional.

Segundo O'Brien e Marakas (2013), os sistemas de informação possuem a função de transformar dados brutos em informações úteis para os gestores, auxiliando no monitoramento das operações, no controle organizacional e no planejamento estratégico das empresas. Nesse contexto, os dados representam registros isolados de eventos ou atividades, enquanto a informação corresponde ao resultado do tratamento e interpretação desses dados, tornando-os relevantes para análise organizacional.

A relação entre dados, informação e conhecimento possui grande relevância dentro da Engenharia de Produção, especialmente em ambientes industriais que dependem de decisões rápidas e fundamentadas. De acordo com Stair e Reynolds (2015), o conhecimento é gerado a partir da interpretação das informações obtidas pelos sistemas organizacionais, permitindo que os gestores compreendam padrões, identifiquem problemas e proponham melhorias nos processos produtivos.

Na Engenharia de Produção, os Sistemas de Informação Gerencial possuem papel fundamental no gerenciamento de atividades relacionadas à produção, logística, estoque, manutenção,

qualidade e cadeia de suprimentos. A utilização eficiente das informações permite que os profissionais acompanhem indicadores de desempenho, reduzam desperdícios e realizem tomadas de decisão de maneira mais assertiva, contribuindo diretamente para o aumento da competitividade organizacional (Slack; Brandon-Jones; Johnston, 2018).

Além disso, os Sistemas de Informação Gerencial estão diretamente relacionados ao pensamento sistêmico, abordagem que considera a organização como um conjunto integrado de processos interdependentes (Chiavenato, 2014). Sob essa perspectiva, as decisões tomadas em determinado setor podem impactar diversas áreas da empresa, tornando fundamental a integração das informações organizacionais para garantir maior alinhamento estratégico e operacional.

Com o avanço da transformação digital e da Indústria 4.0, os sistemas de informação passaram a lidar com volumes cada vez maiores de dados provenientes de sistemas corporativos, sensores industriais, dispositivos conectados e plataformas digitais. Segundo Kagermann, Wahlster e Helbig (2013), a Indústria 4.0 está diretamente associada à utilização intensiva de dados e tecnologias digitais para integração dos processos produtivos, aumentando a necessidade de sistemas capazes de armazenar, processar e disponibilizar informações em tempo real.

Nesse cenário, os bancos de dados assumem papel central na estrutura dos sistemas de informação, sendo responsáveis pelo armazenamento, organização e recuperação das informações utilizadas pelas organizações. Dessa forma, compreender o funcionamento dos SIG e sua relação com os bancos de dados tornou-se uma competência relevante para profissionais da Engenharia de Produção que atuam em ambientes industriais orientados por dados e análise operacional.

2.2 Arquitetura de software

A arquitetura de software pode ser compreendida como a estrutura responsável pela organização dos componentes de um sistema computacional, definindo como esses elementos se relacionam e interagem entre si (Sommerville, 2019). Em sistemas corporativos, a arquitetura possui papel fundamental na separação das responsabilidades do sistema, permitindo maior organização, manutenção, escalabilidade e integração entre os diferentes processos organizacionais.

Segundo Pressman e Maxim (2020), uma arquitetura de software bem estruturada contribui diretamente para a qualidade do sistema, facilitando a manutenção, reutilização de componentes e evolução das aplicações ao longo do tempo. Dessa forma, as organizações conseguem

desenvolver sistemas mais eficientes e alinhados às necessidades operacionais e estratégicas da empresa.

Entre os modelos mais utilizados no desenvolvimento de sistemas empresariais destaca-se a arquitetura em camadas, também conhecida como *layered architecture*. Nesse modelo, o sistema é dividido em diferentes níveis de responsabilidade, normalmente compostos pela camada de apresentação, camada de regras de negócio e camada de acesso a dados (Fowler, 2002). Essa separação permite que alterações realizadas em uma camada tenham menor impacto sobre as demais, aumentando a flexibilidade e a organização do sistema.

Outro modelo amplamente difundido é a arquitetura MVC (Model-View-Controller), utilizada para separar a lógica de negócio, a interface visual e o controle das aplicações (Freeman e Robson, 2004). Nesse padrão, o Model representa os dados e regras do sistema, a View corresponde à interface apresentada ao usuário e o Controller atua como intermediador das requisições realizadas na aplicação. Essa estrutura facilita o desenvolvimento e manutenção dos sistemas, além de melhorar a organização do código-fonte.

Além do MVC, abordagens mais modernas, como a Clean Architecture, também passaram a ser utilizadas em sistemas corporativos e aplicações industriais. De acordo com Martin (2018), a Clean Architecture propõe uma estrutura desacoplada, na qual as regras de negócio permanecem independentes de frameworks, bancos de dados e interfaces externas. Nesse modelo, o banco de dados deixa de ser o centro da aplicação e passa a atuar como uma camada de infraestrutura, permitindo maior flexibilidade tecnológica e facilidade de manutenção.

No contexto da Engenharia de Produção, a arquitetura de software possui importância significativa devido à necessidade de integração entre diferentes sistemas organizacionais, como ERP, sistemas de estoque, controle de produção, logística e qualidade. Segundo Laudon e Laudon (2014), a integração entre sistemas permite maior confiabilidade das informações organizacionais e favorece a tomada de decisão baseada em dados atualizados e consistentes.

Os bancos de dados estão diretamente inseridos nessa arquitetura, sendo responsáveis pelo armazenamento e recuperação das informações utilizadas pelos sistemas empresariais. Dessa forma, compreender como os sistemas são estruturados e como ocorre a comunicação entre aplicação e banco de dados torna-se essencial para profissionais que atuam com análise de dados e suporte à tomada de decisão organizacional.

2.3 Histórico e evolução dos bancos de dados

O desenvolvimento dos bancos de dados está diretamente relacionado à necessidade crescente das organizações em armazenar, organizar e recuperar grandes volumes de informações de maneira eficiente e confiável. Segundo Elmasri e Navathe (2018), os primeiros sistemas utilizados para armazenamento de dados surgiram na década de 1960, baseados em arquivos sequenciais e modelos hierárquicos, nos quais os dados eram organizados em estruturas rígidas e de difícil flexibilização.

Entre os primeiros modelos utilizados destaca-se o modelo hierárquico, implementado em sistemas como o Information Management System (IMS), desenvolvido pela IBM. Nesse modelo, os dados eram organizados em estruturas semelhantes a árvores, nas quais cada registro possuía um único relacionamento pai-filho (Date, 2004). Apesar de apresentar bom desempenho para aplicações específicas, o modelo hierárquico possuía limitações relacionadas à flexibilidade e à complexidade para realização de consultas mais elaboradas.

Na década de 1970, ocorreu uma das principais transformações na área de bancos de dados com a proposição do modelo relacional por Edgar F. Codd. Segundo Codd (1970), os dados poderiam ser organizados em tabelas compostas por linhas e colunas, permitindo maior independência entre os dados e as aplicações. Esse modelo tornou-se um marco na evolução dos sistemas de banco de dados, principalmente pela simplificação da modelagem e pela maior facilidade na realização de consultas e relacionamentos entre informações.

A consolidação do modelo relacional ocorreu principalmente entre as décadas de 1980 e 1990, período em que surgiram diversos Sistemas Gerenciadores de Banco de Dados (SGBD), como Oracle, PostgreSQL, MySQL e SQL Server. Nesse período, a linguagem SQL (Structured Query Language) tornou-se o principal padrão para manipulação e consulta de dados relacionais, permitindo operações como inserção, atualização, exclusão e recuperação de informações de forma padronizada (Silberschatz; Korth; Sudarshan, 2020).

Com o avanço da internet, do comércio eletrônico e das aplicações distribuídas nos anos 2000, os bancos de dados relacionais passaram a enfrentar desafios relacionados à escalabilidade, flexibilidade e processamento de grandes volumes de dados não estruturados. Nesse contexto, surgiram os bancos de dados NoSQL, desenvolvidos para atender aplicações que demandavam alta disponibilidade, escalabilidade horizontal e manipulação de diferentes formatos de dados (Sadalage; Fowler, 2013).

Os bancos NoSQL englobam diferentes modelos de armazenamento, como bancos documentais, chave-valor, orientados a colunas e orientados a grafos. Entre eles, o MongoDB destacou-se pela utilização de documentos no formato JSON/BSON, permitindo maior

flexibilidade estrutural em comparação aos bancos relacionais tradicionais (Banker, 2011). Esse tipo de banco tornou-se amplamente utilizado em aplicações relacionadas à internet das coisas (IoT), análise de logs, sensores industriais e sistemas que necessitam manipular dados semiestruturados.

Atualmente, muitas organizações adotam a estratégia conhecida como polyglot persistence, que consiste na utilização de diferentes tipos de bancos de dados de acordo com as necessidades específicas de cada aplicação (Fowler; Sadalage, 2012). Nesse modelo, bancos relacionais e não relacionais podem coexistir dentro da mesma organização, permitindo combinar consistência transacional, flexibilidade estrutural e escalabilidade operacional.

No contexto da Engenharia de Produção, a evolução dos bancos de dados contribuiu significativamente para o aumento da capacidade de armazenamento e análise das informações organizacionais. Sistemas produtivos modernos passaram a gerar grandes volumes de dados relacionados à produção, manutenção, estoque, logística e qualidade, tornando os bancos de dados elementos essenciais para suporte à tomada de decisão e gestão eficiente das operações industriais

2.4 Bancos de dados relacionais e não relacionais

Os bancos de dados podem ser classificados em diferentes modelos de acordo com a forma como os dados são estruturados, armazenados e recuperados. Entre os principais modelos utilizados atualmente destacam-se os bancos de dados relacionais, baseados em tabelas estruturadas, e os bancos de dados não relacionais (NoSQL), desenvolvidos para aplicações que demandam maior flexibilidade e escalabilidade (Elmasri; Navathe, 2018).

Os bancos de dados relacionais foram desenvolvidos com base no modelo relacional proposto por Edgar F. Codd na década de 1970. Nesse modelo, os dados são organizados em tabelas compostas por linhas e colunas, permitindo o estabelecimento de relacionamentos entre diferentes entidades por meio de chaves primárias e chaves estrangeiras (Date, 2004). Essa estrutura favorece a integridade, a consistência e a padronização das informações armazenadas. Segundo Silberschatz, Korth e Sudarshan (2020), os bancos relacionais utilizam a linguagem SQL (Structured Query Language) como padrão para manipulação e consulta de dados. A linguagem SQL permite realizar operações de inserção, atualização, exclusão e recuperação de informações, além de possibilitar a utilização de JOINS para relacionar tabelas distintas dentro de uma mesma consulta.

Outra característica importante dos bancos relacionais está relacionada ao modelo transacional ACID (Atomicidade, Consistência, Isolamento e Durabilidade), responsável por garantir maior confiabilidade nas operações realizadas no banco de dados (Connolly; Begg, 2015). Essa característica torna os bancos relacionais amplamente utilizados em sistemas corporativos que exigem elevada integridade das informações, como sistemas ERP, controle financeiro, estoque e planejamento da produção.

Entre os principais bancos de dados relacionais utilizados atualmente destacam-se PostgreSQL, MySQL, Oracle Database e Microsoft SQL Server. O PostgreSQL, por exemplo, é amplamente reconhecido pela robustez, estabilidade e suporte a consultas complexas, sendo utilizado em aplicações empresariais e sistemas que demandam alta confiabilidade no armazenamento dos dados (PostgreSQL Global Development Group, 2025).

Por outro lado, os bancos de dados não relacionais, também conhecidos como NoSQL, surgiram como alternativa aos modelos relacionais tradicionais diante do crescimento exponencial do volume de dados gerados pelas aplicações modernas. Segundo Sadalage e Fowler (2013), os bancos NoSQL foram desenvolvidos para oferecer maior escalabilidade horizontal, flexibilidade estrutural e desempenho em aplicações distribuídas.

Diferentemente dos bancos relacionais, os bancos NoSQL não dependem necessariamente de tabelas rígidas e esquemas previamente definidos. No modelo documental, por exemplo, os dados são armazenados em documentos no formato JSON ou BSON, permitindo estruturas mais flexíveis e adaptáveis às necessidades das aplicações (Banker, 2011). Essa abordagem facilita o armazenamento de dados semiestruturados e reduz a necessidade de constantes alterações estruturais no banco de dados.

Entre os principais bancos de dados NoSQL destaca-se o MongoDB, amplamente utilizado em aplicações web, sistemas de monitoramento, internet das coisas (IoT), sensores industriais e armazenamento de logs operacionais. Segundo Chodorow (2019), o MongoDB oferece elevada flexibilidade para manipulação de grandes volumes de dados e permite expansão horizontal por meio da distribuição das informações entre diferentes servidores.

Além das diferenças estruturais, bancos relacionais e não relacionais também apresentam diferenças em relação ao modelo de consistência. Enquanto os bancos relacionais priorizam consistência forte por meio das propriedades ACID, muitos bancos NoSQL utilizam o modelo BASE (Basically Available, Soft State, Eventually Consistent), priorizando disponibilidade e escalabilidade em ambientes distribuídos (Brewer, 2012).

No contexto da Engenharia de Produção, ambos os modelos possuem aplicações relevantes. Bancos relacionais são amplamente utilizados em sistemas ERP, controle de estoque, planejamento e controle da produção, devido à necessidade de integridade e rastreabilidade das informações. Já bancos NoSQL apresentam vantagens em aplicações relacionadas à coleta de dados em tempo real, sensores industriais, monitoramento de máquinas e análise de grandes volumes de dados operacionais.

Dessa forma, a escolha entre bancos relacionais e não relacionais depende diretamente das características da aplicação, do volume de dados, da necessidade de escalabilidade e do tipo de informação manipulada. Atualmente, muitas organizações adotam arquiteturas híbridas, utilizando diferentes tecnologias de bancos de dados de forma complementar para atender demandas específicas dos sistemas organizacionais.

2.5 Normalização de dados e relacionamentos

A normalização de dados consiste em um processo de organização das informações em bancos de dados relacionais com o objetivo de reduzir redundâncias, evitar inconsistências e garantir maior integridade dos dados armazenados (Elmasri; Navathe, 2018). Segundo Date (2004), a normalização contribui para melhorar a eficiência estrutural do banco de dados, facilitando operações de inserção, atualização e remoção de informações.

De acordo com Silberschatz, Korth e Sudarshan (2020), o processo de normalização é dividido em diferentes etapas conhecidas como formas normais, sendo as três primeiras formas normais as mais utilizadas em sistemas corporativos. Essas etapas possuem como finalidade organizar adequadamente os atributos das tabelas e os relacionamentos existentes entre as entidades do banco de dados.

2.5.1 Primeira forma normal (1FN)

A Primeira Forma Normal (1FN) estabelece que cada atributo da tabela deve possuir apenas valores atômicos, ou seja, indivisíveis, eliminando listas, grupos repetitivos ou múltiplos valores em uma mesma célula (Connolly; Begg, 2015). Em ambientes industriais, um exemplo de violação da 1FN ocorre quando uma tabela de ordens de produção armazena diversos códigos de produtos em um único campo, dificultando consultas e manipulação dos dados.

2.5.2 Segunda forma normal (2FN)

A Segunda Forma Normal (2FN) determina que todos os atributos não pertencentes à chave primária devem depender totalmente da chave primária completa, eliminando dependências parciais (Date, 2004). Esse problema normalmente ocorre em tabelas que utilizam chaves compostas, nas quais determinados atributos dependem apenas de parte da chave principal. A aplicação da 2FN contribui para redução de redundâncias e inconsistências nos dados armazenados.

2.5.3 Terceira forma normal (3FN)

Já a Terceira Forma Normal (3FN) estabelece que atributos não pertencentes à chave primária não devem depender de outros atributos não chave, eliminando dependências transitivas (Elmasri; Navathe, 2018). Em um cenário industrial, isso pode ocorrer quando informações de fornecedores são armazenadas diretamente em tabelas de produtos, gerando repetição desnecessária de dados e maior risco de inconsistências durante atualizações.

Além da normalização, os relacionamentos entre tabelas representam elemento fundamental na modelagem de bancos de dados relacionais. Segundo Coronel e Morris (2016), os relacionamentos permitem representar associações existentes entre diferentes entidades organizacionais, contribuindo para maior integridade e organização das informações armazenadas.

2.5.4 Relacionamento 1:1

O relacionamento do tipo um para um (1:1) ocorre quando um registro de uma tabela está associado exclusivamente a um único registro de outra tabela. Um exemplo aplicado à Engenharia de Produção seria a relação entre funcionário e crachá, em que cada colaborador possui apenas um crachá vinculado ao seu cadastro (Silberschatz, Korth e Sudarshan, 2020).

2.5.5 Relacionamento 1:N

O relacionamento um para muitos (1:N) ocorre quando um registro de uma entidade pode estar relacionado a vários registros de outra entidade. Em sistemas industriais, um setor pode possuir diversos funcionários associados, enquanto cada funcionário pertence a apenas um setor específico (Connolly e Begg, 2015). Esse tipo de relacionamento é amplamente utilizado em sistemas ERP e bancos de dados corporativos.

2.5.6 Relacionamento N:M

Já o relacionamento muitos para muitos (N:M) ocorre quando múltiplos registros de uma entidade podem se relacionar com múltiplos registros de outra entidade. Segundo Elmasri e Navathe (2018), esse tipo de relacionamento normalmente exige a criação de uma tabela associativa intermediária. Um exemplo comum na Engenharia de Produção corresponde à relação entre produtos e fornecedores, na qual um fornecedor pode fornecer diversos produtos e um mesmo produto pode possuir múltiplos fornecedores cadastrados.

A correta aplicação das formas normais e dos relacionamentos contribui significativamente para a qualidade estrutural do banco de dados, favorecendo a integridade, rastreabilidade e confiabilidade das informações organizacionais. Dessa forma, a modelagem adequada dos dados torna-se essencial para garantir maior eficiência nas análises realizadas pelos sistemas de informação utilizados nas organizações.

2.6 Agregações e composições na modelagem de dados

A modelagem de dados possui papel fundamental na organização das informações em sistemas computacionais, permitindo representar entidades, atributos e relacionamentos de maneira estruturada e coerente com os processos organizacionais (Elmasri; Navathe, 2018). Nesse contexto, os conceitos de agregação e composição são utilizados para representar diferentes níveis de dependência entre os elementos existentes em um sistema.

Segundo Booch, Rumbaugh e Jacobson (2005), a agregação representa uma relação do tipo “tem um”, na qual os objetos associados possuem existência independente entre si. Nesse modelo, mesmo que a entidade principal deixe de existir, os elementos relacionados continuam existindo de forma autônoma dentro do sistema.

Na Engenharia de Produção, um exemplo de agregação pode ser observado na relação entre fornecedores e pedidos de compra. Embora um pedido esteja associado a determinado fornecedor, o fornecedor continua existindo no banco de dados mesmo que o pedido seja removido ou cancelado. Esse tipo de modelagem é amplamente utilizado em sistemas ERP e aplicações corporativas que trabalham com entidades independentes (Pressman; Maxim, 2020). Em bancos de dados relacionais, a agregação normalmente é implementada por meio de relacionamentos entre tabelas utilizando chaves estrangeiras e operações de JOIN para recuperação das informações relacionadas (Silberschatz; Korth; Sudarshan, 2020). Já em bancos de dados NoSQL, como o MongoDB, esse relacionamento pode ocorrer por meio de referências entre documentos utilizando identificadores do tipo ObjectId (Chodorow, 2019).

Por outro lado, a composição representa uma relação de dependência forte entre os elementos, caracterizando uma estrutura do tipo “parte de” (Booch; Rumbaugh; Jacobson, 2005). Nesse modelo, os elementos subordinados não possuem existência independente da entidade principal, sendo diretamente dependentes dela para existir dentro do sistema.

Um exemplo aplicado à Engenharia de Produção pode ser observado na relação entre ordens de produção e seus respectivos itens. Os itens pertencem exclusivamente à ordem de produção e deixam de existir caso a ordem seja removida do sistema. Esse tipo de modelagem é importante para garantir consistência e integridade das informações relacionadas aos processos produtivos (Slack; Brandon-Jones; Johnston, 2018).

Em bancos de dados relacionais, a composição geralmente é implementada utilizando tabelas filhas associadas por chaves estrangeiras com regras de exclusão em cascata, como o comando ON DELETE CASCADE (Connolly; Begg, 2015). Dessa forma, ao excluir o registro principal, todos os registros dependentes também são removidos automaticamente, preservando a integridade referencial do banco de dados.

Nos bancos de dados NoSQL, especialmente em bancos documentais como o MongoDB, a composição costuma ser representada por documentos embutidos (embedded documents), nos quais as informações subordinadas são armazenadas dentro do próprio documento principal (Banker, 2011). Essa abordagem favorece desempenho e simplifica consultas em cenários onde os dados possuem forte dependência estrutural.

A compreensão dos conceitos de agregação e composição é importante para a correta modelagem dos bancos de dados utilizados nas organizações, permitindo estruturar adequadamente os relacionamentos entre entidades e garantir maior eficiência no armazenamento e recuperação das informações. Além disso, esses conceitos contribuem para desenvolvimento de sistemas mais organizados, consistentes e alinhados às necessidades operacionais da Engenharia de Produção

3. Metodologia

3.1 Caracterização da pesquisa

O presente trabalho caracteriza-se como uma pesquisa de natureza aplicada, uma vez que busca utilizar conhecimentos teóricos relacionados a sistemas de informação, arquitetura de software e bancos de dados para elaboração de um Procedimento Operacional Padrão (POP) voltado ao contexto da Engenharia de Produção. O objetivo principal consiste em documentar o processo

de acesso, conexão, extração e análise de dados diretamente em bancos de dados utilizados em ambientes organizacionais.

Quanto aos objetivos, a pesquisa possui caráter exploratório e descritivo. Segundo Prodanov e Freitas (2013), pesquisas exploratórias possuem como finalidade proporcionar maior familiaridade com determinado problema, enquanto pesquisas descritivas buscam registrar, analisar e descrever características de fenômenos ou processos. Nesse contexto, o trabalho explora os conceitos relacionados aos bancos de dados e descreve os procedimentos necessários para realização das conexões e extração de informações.

A abordagem metodológica adotada possui caráter qualitativo e quantitativo. A abordagem qualitativa está relacionada à interpretação dos conceitos teóricos, modelagem de dados e análise dos procedimentos utilizados nos sistemas de informação. Já a abordagem quantitativa está associada à manipulação e análise de dados extraídos dos bancos de dados por meio de consultas estruturadas e técnicas computacionais.

Além disso, o trabalho foi estruturado no formato de artigo científico associado a um Procedimento Operacional Padrão (POP), conforme proposta apresentada na disciplina. Segundo Campos (1992), o POP possui como finalidade padronizar procedimentos operacionais, garantindo repetibilidade, organização e confiabilidade na execução das atividades descritas. Dessa forma, o presente artigo busca documentar de maneira clara e reproduzível os procedimentos relacionados ao acesso e análise de dados em bancos de dados aplicados à Engenharia de Produção.

A pesquisa utilizará como base bancos de dados relacionais e não relacionais disponibilizados para a disciplina, permitindo a análise comparativa entre diferentes modelos de armazenamento de dados. Para isso, serão abordados procedimentos de conexão utilizando PostgreSQL e MongoDB, além da utilização de ferramentas e bibliotecas computacionais voltadas à extração e manipulação das informações.

3.2 Tecnologias e ferramentas utilizadas

Para desenvolvimento do presente trabalho serão utilizados bancos de dados relacionais e não relacionais, possibilitando a aplicação prática dos conceitos abordados ao longo da fundamentação teórica. Segundo Elmasri e Navathe (2018), a utilização de diferentes modelos de bancos de dados permite compreender características específicas relacionadas à estrutura, escalabilidade e organização das informações.

O banco de dados relacional utilizado será o PostgreSQL, Sistema Gerenciador de Banco de Dados (SGBD) amplamente reconhecido pela robustez, confiabilidade e suporte a operações transacionais complexas (PostgreSQL Global Development Group, 2025). O PostgreSQL será utilizado para representar ambientes estruturados que exigem integridade referencial, normalização e relacionamento entre tabelas.

Para o contexto de banco de dados não relacional será utilizado o MongoDB, banco documental orientado a documentos JSON/BSON, amplamente utilizado em aplicações que demandam flexibilidade estrutural e escalabilidade horizontal (Chodorow, 2019). O MongoDB permitirá explorar conceitos relacionados ao armazenamento de dados semiestruturados e utilização de coleções documentais.

As conexões e manipulações dos bancos de dados serão realizadas utilizando a linguagem Python, devido à sua ampla utilização em aplicações voltadas para análise de dados, automação e integração com sistemas computacionais (Matthes, 2023). Além disso, serão utilizadas bibliotecas específicas para comunicação com os bancos de dados, permitindo execução de consultas e manipulação das informações extraídas.

Para conexão com o PostgreSQL será utilizada a biblioteca psycopg2, responsável por estabelecer comunicação entre aplicações Python e bancos PostgreSQL (Psycopg Development Team, 2025). Já para conexão com o MongoDB será utilizada a biblioteca pymongo, desenvolvida para interação entre aplicações Python e bancos documentais MongoDB (MongoDB Inc., 2025).

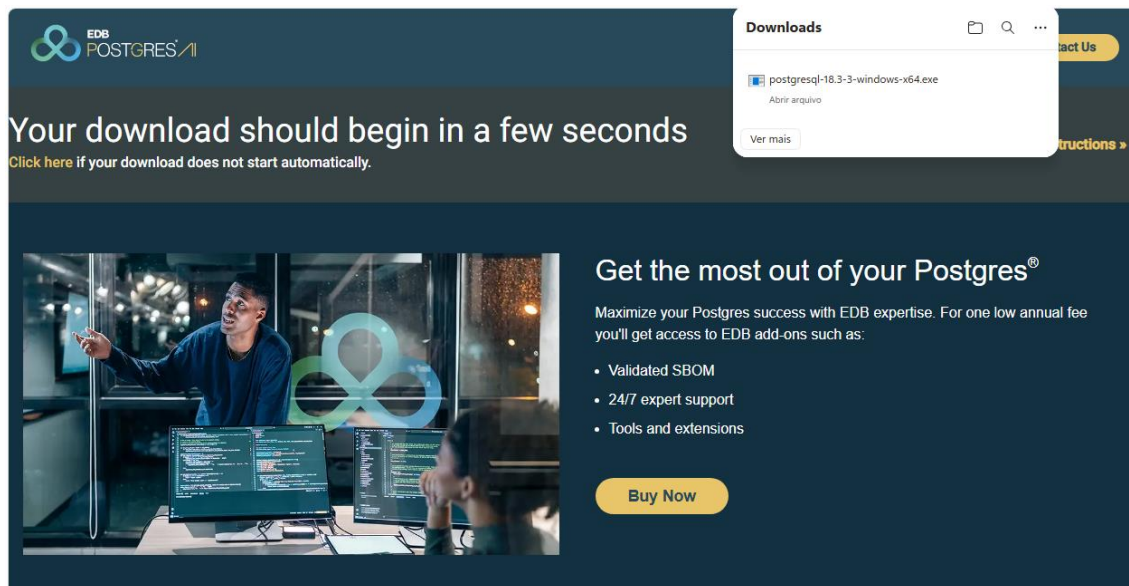
Além disso, será utilizada a biblioteca pandas para organização, tratamento e análise dos dados extraídos dos bancos de dados. Segundo McKinney (2022), o pandas é uma das principais bibliotecas utilizadas em ciência de dados e análise computacional devido à sua capacidade de manipulação eficiente de estruturas tabulares e datasets complexos.

As conexões gráficas com os bancos de dados serão realizadas utilizando as ferramentas pgAdmin e MongoDB Atlas/MongoDB Compass. O pgAdmin será utilizado para administração e visualização das tabelas relacionais do PostgreSQL, enquanto o MongoDB Atlas/MongoDB Compass será empregado para exploração das coleções documentais e análise estrutural do MongoDB (MongoDB Inc., 2025).

3.3 Procedimento operacional para conexão com PostgreSQL

O procedimento inicia-se com a instalação do PostgreSQL no computador. Para isso, o usuário deverá acessar o site oficial do PostgreSQL e realizar o download da versão compatível com o sistema operacional utilizado, como mostra na figura 1.

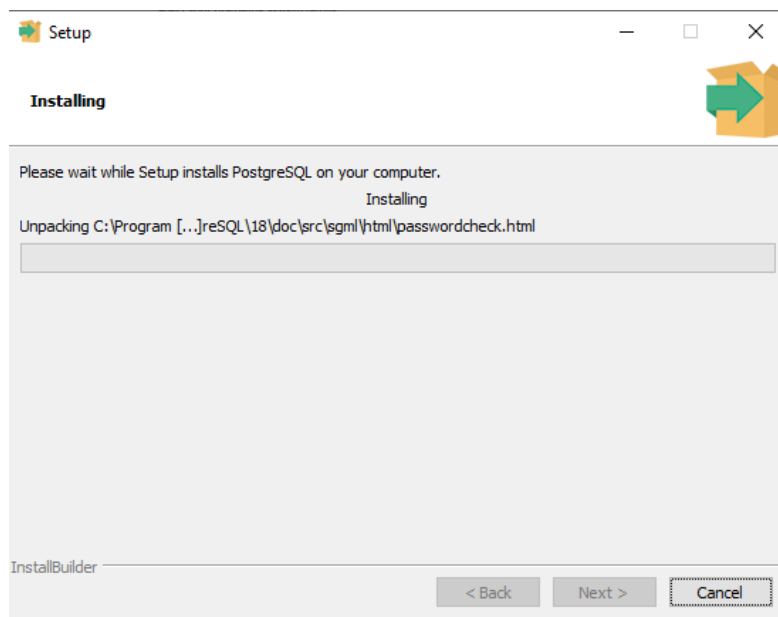
Figura 1 – Página para download do PostgreSQL



Fonte: PostgreSQL (2026).

Após concluir o download, deverá executar o instalador e seguir as instruções apresentadas na tela. Durante a instalação, recomenda-se manter os componentes padrão selecionados, incluindo PostgreSQL Server, pgAdmin e Command Line Tools.

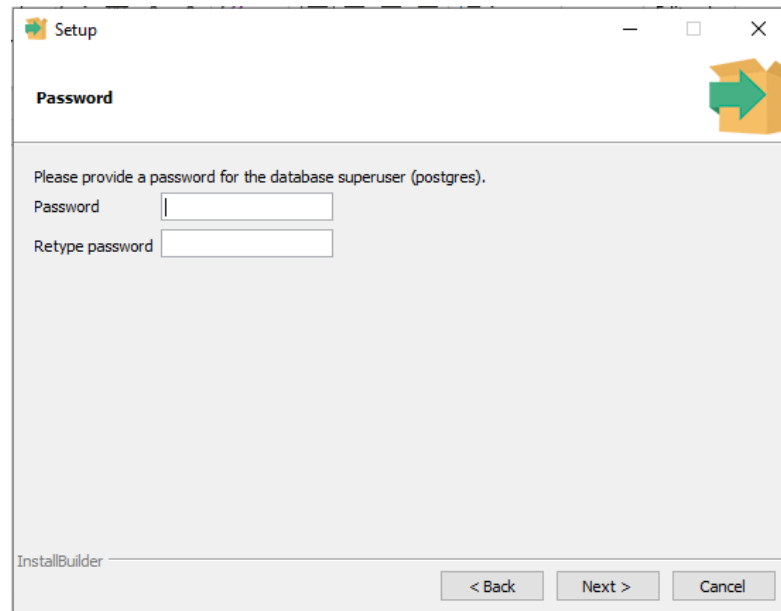
Figura 2 – Instalação do programa



Fonte: PostgreSQL (2026).

Em uma das etapas da instalação, o sistema solicitará a criação de uma senha para o usuário padrão denominado “postgres”.

Figura 3 - Criação de senha para usuário

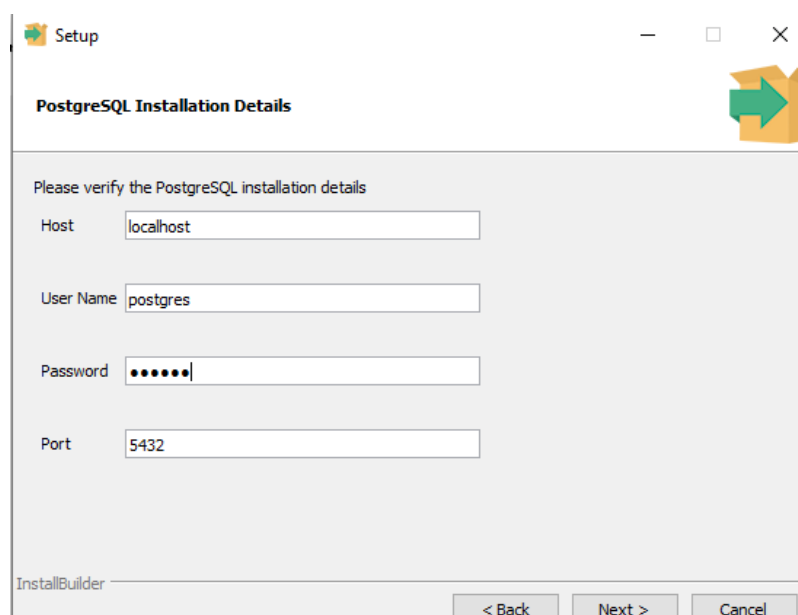


The screenshot shows a 'Setup' window titled 'Password'. It contains the text 'Please provide a password for the database superuser (postgres).' followed by two input fields: 'Password' and 'Retype password'. At the bottom, there are three buttons: '< Back', 'Next >', and 'Cancel'. The 'InstallBuilder' logo is visible in the bottom left corner.

Fonte: PostgreSQL (2026).

Essa senha deverá ser armazenada, pois será utilizada posteriormente durante as conexões com o banco de dados. Recomenda-se também manter a porta padrão “5432”, utilizada nativamente pelo PostgreSQL para comunicação entre aplicações e o servidor do banco de dados. Segundo Elmasri e Navathe (2018), a configuração adequada do ambiente é fundamental para garantir estabilidade e funcionamento correto do banco relacional.

Figura 4 – Detalhes para instalação

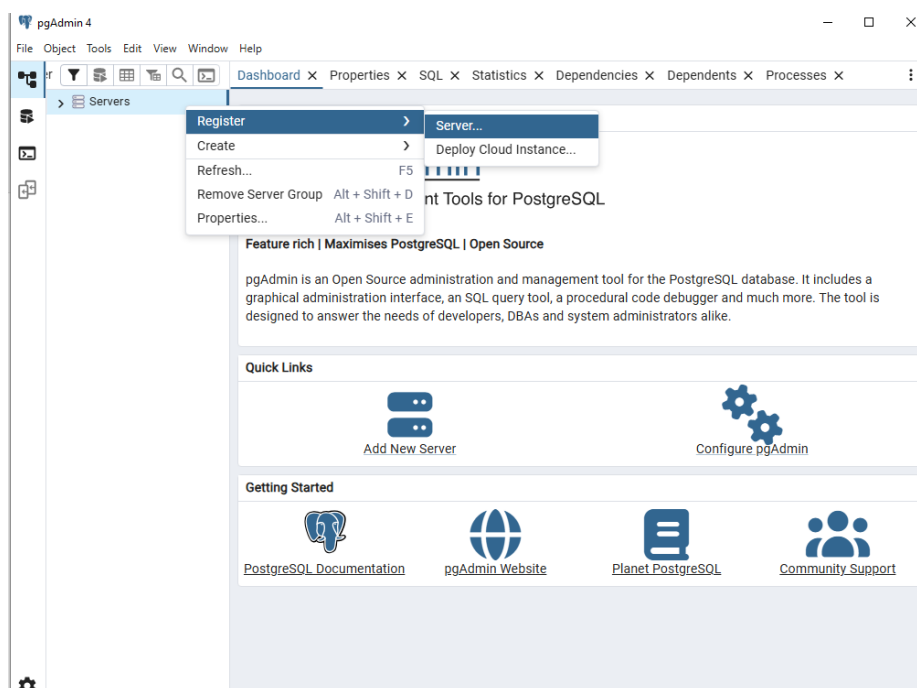


The screenshot shows a 'Setup' window titled 'PostgreSQL Installation Details'. It contains the text 'Please verify the PostgreSQL installation details' followed by four input fields: 'Host' (with 'localhost' entered), 'User Name' (with 'postgres' entered), 'Password' (with masked characters '•••••' entered), and 'Port' (with '5432' entered). At the bottom, there are three buttons: '< Back', 'Next >', and 'Cancel'. The 'InstallBuilder' logo is visible in the bottom left corner.

Fonte: PostgreSQL (2026).

Após finalizar a instalação, o usuário deverá abrir o pgAdmin, ferramenta gráfica utilizada para administração do PostgreSQL. Ao abrir o sistema pela primeira vez, será necessário registrar uma conexão com o servidor local instalado anteriormente. Para isso, o usuário deverá localizar a seção “Servers”, clicar com o botão direito do mouse e selecionar a opção “Register” seguida de “Server”.

Figura 5 – Registrando conexão com servidor

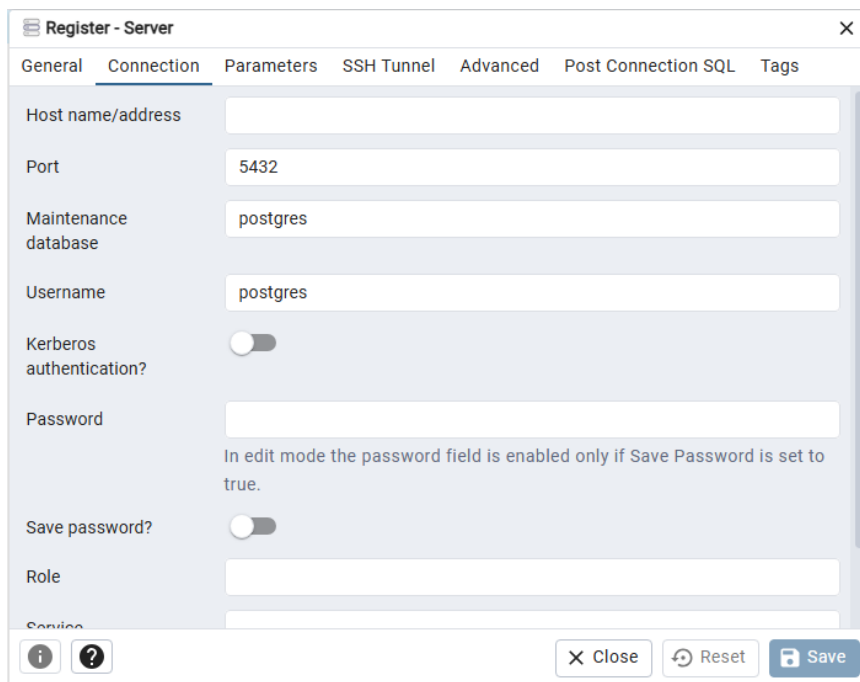


Fonte: PostgreSQL (2026).

Na janela de configuração da conexão, deverá ser definido um nome para identificação do servidor. Em seguida, na aba “Connection”, deverão ser preenchidos os seguintes parâmetros:

- Host: localhost ou endereço IP
- Port: 5432
- Maintenance database: Nome do Banco
- Username: postgres
- Password: senha definida durante a instalação

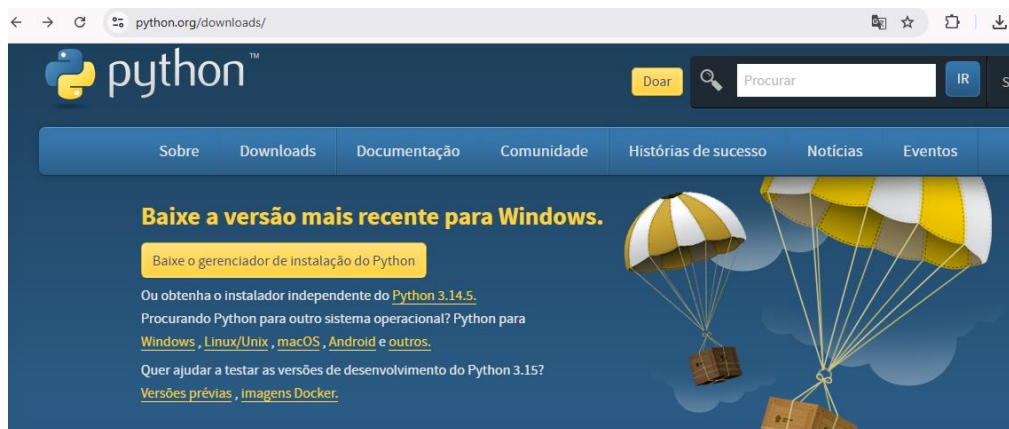
Figura 6 – Preenchimento de parâmetros



Fonte: PostgreSQL (2026).

Após preencher as informações, o usuário deverá clicar em “Save”. Caso a configuração tenha sido realizada corretamente, o servidor será exibido no painel lateral esquerdo do pgAdmin. Segundo PostgreSQL Global Development Group (2025), o pgAdmin permite executar comandos SQL, visualizar tabelas e administrar bancos relacionais de maneira visual e intuitiva. Concluída a configuração do PostgreSQL, será necessária a instalação da linguagem Python, utilizada para realização das conexões programáticas e análises dos dados. Para isso, o usuário deverá acessar o site oficial do Python, realizar o download da versão mais recente e executar o instalador.

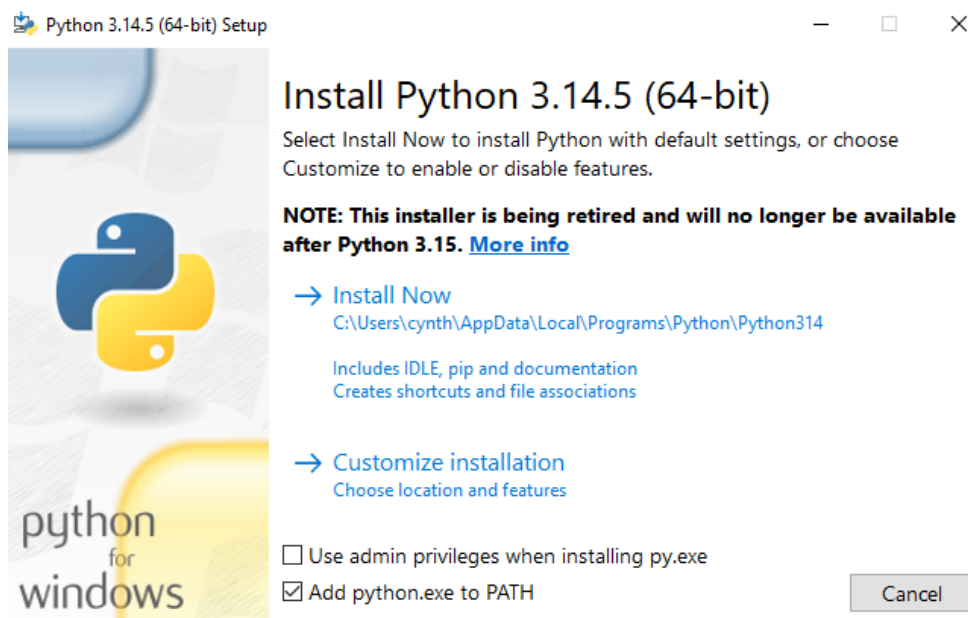
Figura 7 – Página para download do Python



Fonte: Python (2026).

Durante a instalação, recomenda-se marcar a opção “Add Python to PATH”, permitindo que o Python seja reconhecido automaticamente pelo sistema operacional.

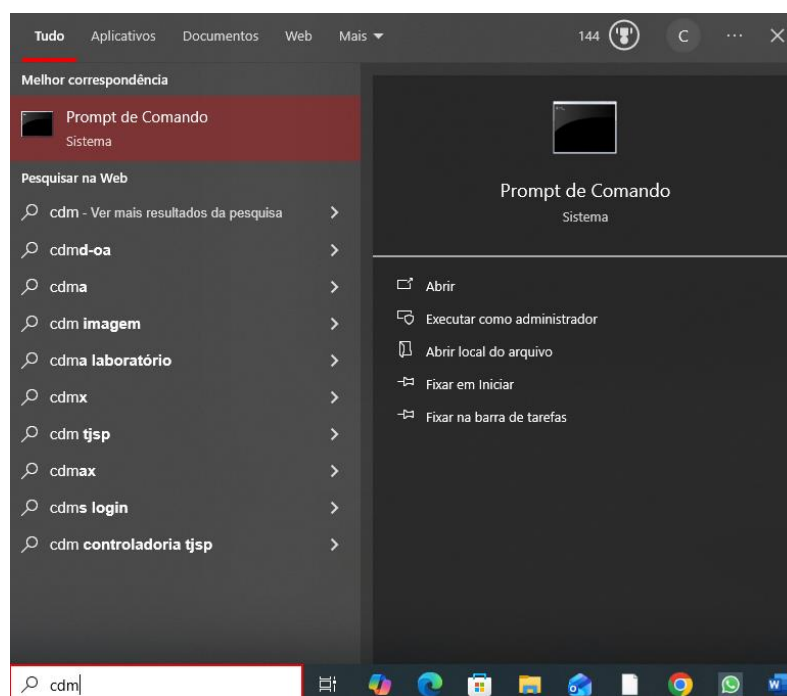
Figura 8 – Opção “Add Python to PATH” marcada no instalador



Fonte: Python (2026).

Após instalação do Python, o próximo passo consiste na instalação das bibliotecas responsáveis pela comunicação entre a aplicação e o banco de dados PostgreSQL. O usuário deverá abrir o Prompt de Comando ou Terminal do sistema operacional e executar os seguintes comandos:

Figura 9 – Abrindo o Prompt de Comando



Fonte: Elaborado pelo autor (2026).

Figura 10 – Instalando biblioteca psycpg2

```
Ca. Selecionar Prompt de Comando - pip install pandas

(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\cynth>pip install psycpg2
Collecting psycpg2
  Downloading psycpg2-2.9.12-cp314-cp314-win_amd64.whl.metadata (5.1 kB)
  Downloading psycpg2-2.9.12-cp314-cp314-win_amd64.whl (2.8 MB)
----- 2.8/2.8 MB 9.8 MB/s 0:00:00
Installing collected packages: psycpg2
Successfully installed psycpg2-2.9.12
```

Fonte: Elaborado pelo autor (2026).

Figura 11 – Instalando biblioteca pandas

```
Ca. Prompt de Comando

C:\Users\cynth>pip install pandas
Collecting pandas
  Downloading pandas-3.0.2-cp314-cp314-win_amd64.whl.metadata (19 kB)
Collecting numpy>=2.3.3 (from pandas)
  Downloading numpy-2.4.4-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting tzdata (from pandas)
  Downloading tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
  Downloading pandas-3.0.2-cp314-cp314-win_amd64.whl (9.9 MB)
----- 9.9/9.9 MB 10.1 MB/s 0:00:01
  Downloading numpy-2.4.4-cp314-cp314-win_amd64.whl (12.4 MB)
----- 12.4/12.4 MB 10.8 MB/s 0:00:01
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
  Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Installing collected packages: tzdata, six, numpy, python-dateutil, pandas
Successfully installed numpy-2.4.4 pandas-3.0.2 python-dateutil-2.9.0.post0 six-1.17.0 tzdata-2026.2
```

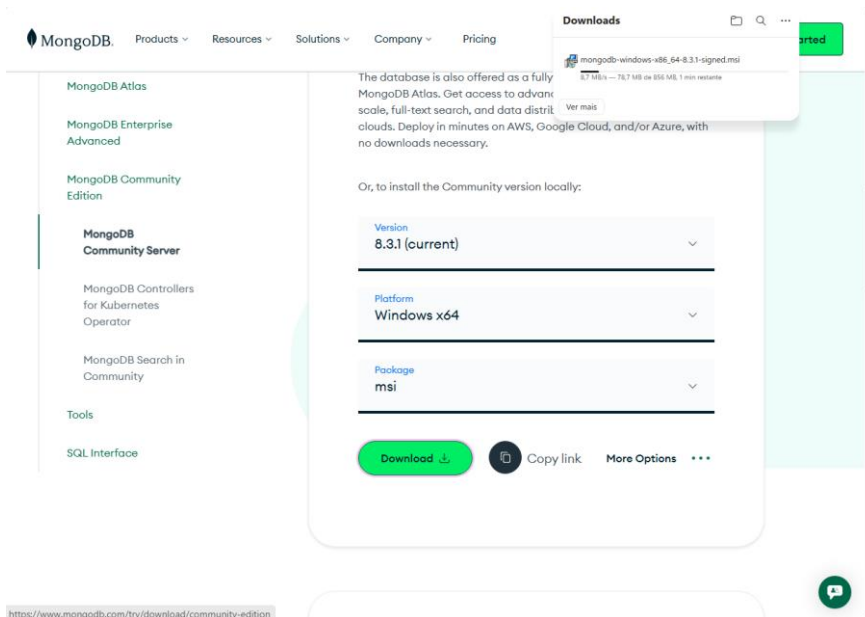
Fonte: Elaborado pelo autor (2026).

A biblioteca psycpg2 será responsável pela conexão entre Python e PostgreSQL, enquanto a biblioteca pandas será utilizada para organização e manipulação dos dados extraídos do banco de dados (Psycpg Development Team, 2025).

3.4 Procedimento operacional para conexão com MongoDB

O procedimento inicia-se com o download do MongoDB. Para isso, o usuário deverá acessar o site oficial do MongoDB e selecionar a versão compatível com o sistema operacional utilizado.

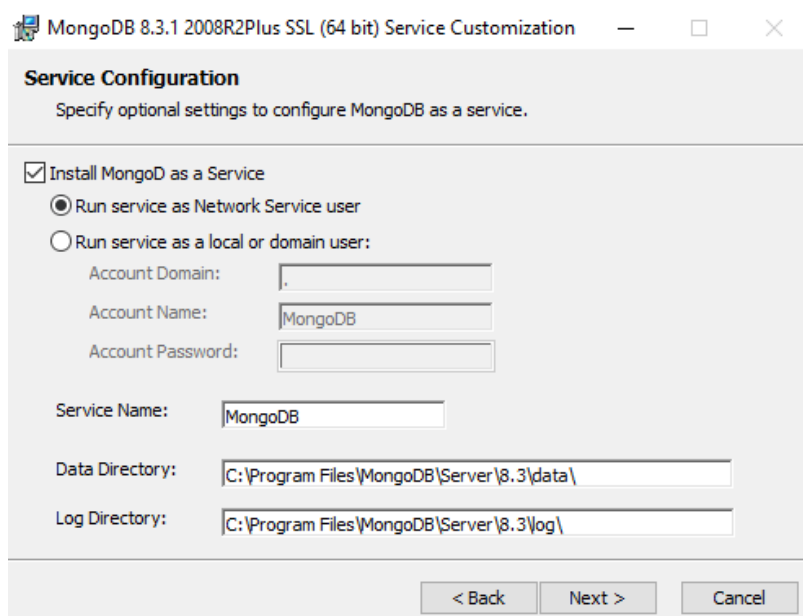
Figura 12 – Página para download do MongoDB



Fonte: MongoDB (2026).

Após concluir o download, deverá executar o instalador e seguir as etapas apresentadas na tela, mantendo as configurações padrão recomendadas pelo sistema.

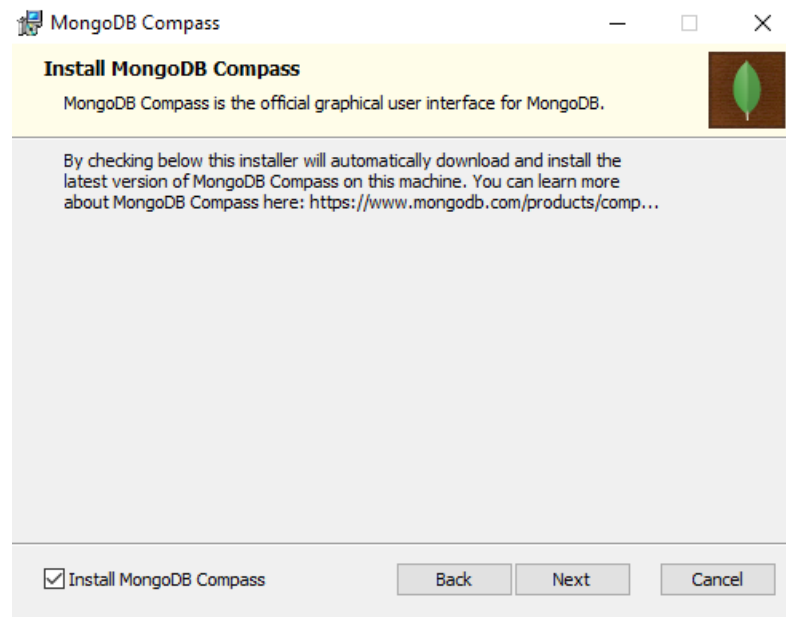
Figura 13 – Detalhes para instalação



Fonte: MongoDB (2026).

Durante a instalação, recomenda-se manter selecionadas as opções relacionadas ao MongoDB Server e ao MongoDB Compass.

Figura 14 – Opção “*Install MongoDB Compass*” marcada no instalador



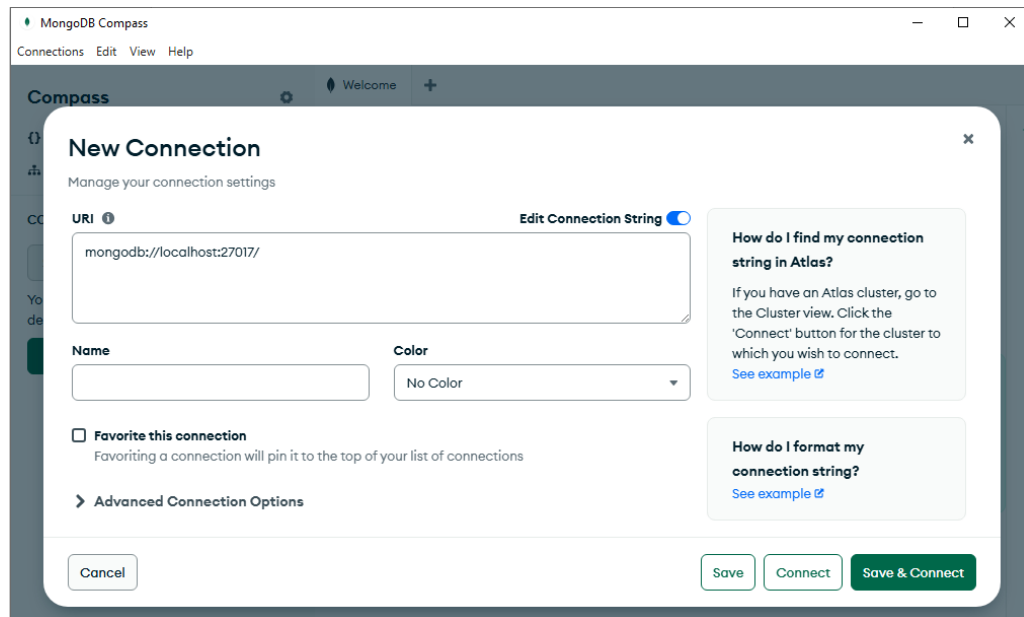
Fonte: MongoDB (2026).

O MongoDB Server será responsável pelo funcionamento do banco de dados, enquanto o MongoDB Compass será utilizado como ferramenta gráfica para visualização das coleções, documentos e bases de dados armazenadas no sistema. Segundo Banker (2011), ferramentas gráficas contribuem para facilitar a administração e exploração dos dados em ambientes não relacionais.

Após finalizar a instalação, o usuário deverá verificar se o serviço do MongoDB se encontra em execução no sistema operacional. Em muitos casos, o serviço é iniciado automaticamente após a instalação. Caso isso não ocorra, o usuário deverá acessar o gerenciador de serviços do sistema operacional e localizar o serviço denominado “MongoDB Server”. Em seguida, deverá clicar na opção “Iniciar”.

Com o servidor ativo, o próximo passo consiste em abrir o MongoDB Compass para realização da conexão gráfica com o banco de dados. Ao iniciar o programa, será exibido um botão denominado “*New Connection*”. A figura 15 mostra o botão mencionado.

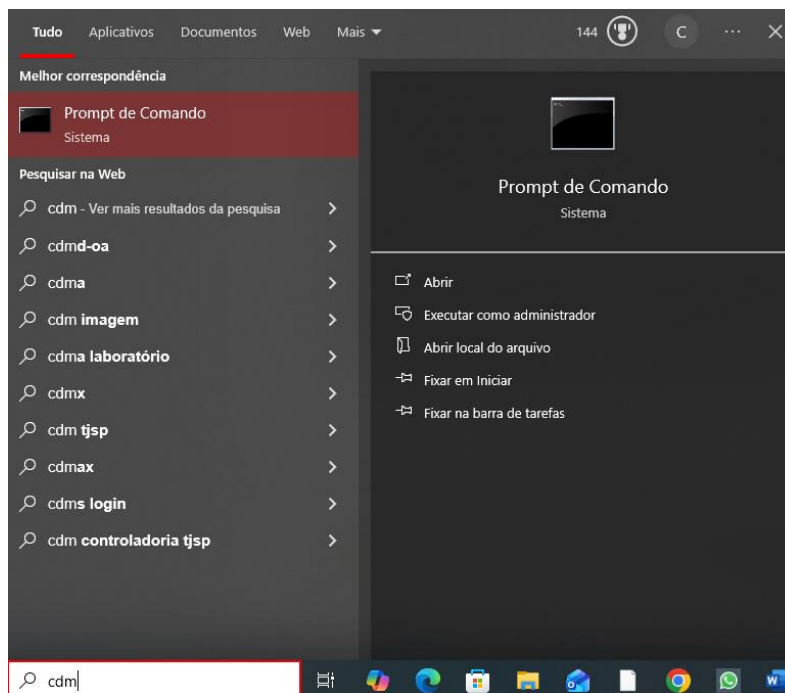
Figura 15 – Botão “*New Connection*”



Fonte: MongoDB (2026).

Após verificar o endereço, o usuário deverá clicar no botão “Connect”. Caso a conexão seja realizada corretamente, as bases de dados disponíveis serão exibidas no painel lateral esquerdo da aplicação. Segundo MongoDB Inc. (2025), a porta “27017” corresponde à porta padrão utilizada pelo MongoDB para comunicação entre aplicações e o servidor do banco de dados. Após estabelecimento da conexão gráfica, será necessária a instalação das bibliotecas utilizadas para comunicação entre Python e MongoDB. Para isso, o usuário deverá abrir o Prompt de Comando ou Terminal do sistema operacional e executar os seguintes comandos:

Figura 16 – Abrindo o Prompt de Comando



Fonte: Elaborado pelo autor (2026).

Figura 17 – Instalando biblioteca pymongo

```
C:\> Seleccionar Prompt de Comando - pip install pandas

(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\cynth>pip install psycpg2
Collecting psycpg2
  Downloading psycpg2-2.9.12-cp314-cp314-win_amd64.whl.metadata (5.1 kB)
  Downloading psycpg2-2.9.12-cp314-cp314-win_amd64.whl (2.8 MB)
----- 2.8/2.8 MB 9.8 MB/s 0:00:00
Installing collected packages: psycpg2
Successfully installed psycpg2-2.9.12
```

Fonte: Elaborado pelo autor (2026).

Figura 18 – Instalando biblioteca pandas

```
C:\> Prompt de Comando

C:\Users\cynth>pip install pandas
Collecting pandas
  Downloading pandas-3.0.2-cp314-cp314-win_amd64.whl.metadata (19 kB)
Collecting numpy>=2.3.3 (from pandas)
  Downloading numpy-2.4.4-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting tzdata (from pandas)
  Downloading tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading pandas-3.0.2-cp314-cp314-win_amd64.whl (9.9 MB)
----- 9.9/9.9 MB 10.1 MB/s 0:00:01
Downloading numpy-2.4.4-cp314-cp314-win_amd64.whl (12.4 MB)
----- 12.4/12.4 MB 10.8 MB/s 0:00:01
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Installing collected packages: tzdata, six, numpy, python-dateutil, pandas
Successfully installed numpy-2.4.4 pandas-3.0.2 python-dateutil-2.9.0.post0 six-1.17.0 tzdata-2026.2
```

Fonte: Elaborado pelo autor (2026).

A biblioteca pymongo será responsável pela integração entre Python e MongoDB, permitindo execução de consultas, recuperação de documentos e manipulação das coleções armazenadas no banco de dados. Já a biblioteca pandas será utilizada para organização e tratamento das informações extraídas (MongoDB Inc., 2025).

Após estabelecer conexão com o servidor, será possível acessar bases de dados e coleções específicas. Diferentemente dos bancos relacionais, o MongoDB trabalha com documentos organizados em coleções, não utilizando tabelas estruturadas com linhas e colunas (Chodorow, 2019). Essa característica proporciona maior flexibilidade no armazenamento dos dados.

4. Considerações finais

O desenvolvimento do presente trabalho possibilitou compreender a importância dos sistemas de informação e dos bancos de dados no contexto organizacional e na Engenharia de Produção. A partir da fundamentação teórica realizada, foi possível analisar conceitos relacionados à arquitetura de software, evolução dos bancos de dados, modelos relacionais e não relacionais, normalização, relacionamentos e estruturas de modelagem utilizadas no armazenamento e gerenciamento das informações organizacionais.

Além disso, o trabalho permitiu identificar como os bancos de dados se tornaram elementos essenciais para suporte à tomada de decisão nas organizações modernas. Segundo Laudon e Laudon (2014), a capacidade de coletar, armazenar e transformar dados em informações relevantes tornou-se um diferencial estratégico para empresas que buscam maior eficiência operacional e competitividade no mercado.

A elaboração dos procedimentos operacionais relacionados ao PostgreSQL e ao MongoDB contribuiu para transformar os conceitos teóricos em aplicações práticas e reproduzíveis. A documentação detalhada das etapas de instalação, configuração, conexão e realização de consultas permitiu estruturar um material capaz de orientar usuários mesmo sem experiência avançada em bancos de dados, aproximando o trabalho das características de um Procedimento Operacional Padrão (POP).

Outro ponto relevante observado durante o desenvolvimento do trabalho foi a diferença estrutural entre bancos de dados relacionais e não relacionais. Enquanto o PostgreSQL demonstrou maior foco em integridade, relacionamentos e consistência das informações, o MongoDB apresentou maior flexibilidade estrutural para manipulação de dados semiestruturados. Dessa forma, foi possível compreender que a escolha do banco de dados

depende diretamente das necessidades operacionais e das características das aplicações utilizadas pelas organizações.

No contexto da Engenharia de Produção, os conhecimentos abordados ao longo do trabalho possuem grande relevância devido à crescente utilização de dados em processos produtivos, sistemas ERP, monitoramento industrial, logística, controle de estoque e análise de desempenho operacional. A capacidade de acessar diretamente a fonte dos dados e transformar informações em suporte para tomada de decisão representa uma competência cada vez mais importante para profissionais da área.

Além disso, a construção do presente artigo permitiu desenvolver uma visão mais integrada entre tecnologia, gestão e análise de dados, demonstrando que o profissional de Engenharia de Produção necessita compreender não apenas os processos organizacionais, mas também as ferramentas computacionais responsáveis pelo armazenamento e manipulação das informações utilizadas pelas empresas.

Por fim, destaca-se que o trabalho desenvolvido nesta primeira etapa servirá como base para as próximas fases da disciplina, nas quais serão realizadas aplicações práticas utilizando os bancos de dados disponibilizados pelo professor. Dessa forma, espera-se que os procedimentos documentados contribuam para facilitar futuras etapas de extração, análise e interpretação dos dados organizacionais.

REFERÊNCIAS

- BANKER, K. MongoDB in action. New York: Manning Publications, 2011.
- BOOCH, G.; RUMBAUGH, J.; JACOBSON, I. UML: guia do usuário. 2. ed. Rio de Janeiro: Elsevier, 2005.
- BREWER, E. CAP twelve years later: how the "rules" have changed. Computer, Los Alamitos, v. 45, n. 2, p. 23-29, 2012.
- CHIAVENATO, I. Introdução à teoria geral da administração. 9. ed. Barueri: Manole, 2014.
- CHODOROW, K. MongoDB: the definitive guide. 3. ed. Sebastopol: O'Reilly Media, 2019.
- CODD, E. F. A relational model of data for large shared data banks. Communications of the ACM, New York, v. 13, n. 6, p. 377-387, 1970.
- CONNOLLY, T.; BEGG, C. Database systems: a practical approach to design, implementation and management. 6. ed. Boston: Pearson, 2015.
- CORONEL, C.; MORRIS, S. Database systems: design, implementation and management. 12. ed. Boston: Cengage Learning, 2016.
- DATE, C. J. Introdução a sistemas de bancos de dados. 8. ed. Rio de Janeiro: Elsevier, 2004.
- ELMASRI, R.; NAVATHE, S. B. Sistemas de banco de dados. 7. ed. São Paulo: Pearson, 2018.
- FOWLER, M. Patterns of enterprise application architecture. Boston: Addison-Wesley, 2002.
- FOWLER, M.; SADALAGE, P. J. NoSQL distilled: a brief guide to the emerging world of polyglot persistence. Boston: Addison-Wesley, 2012.
- FREEMAN, E.; ROBSON, E. Head first design patterns. Sebastopol: O'Reilly Media, 2004.
- KAGERMANN, H.; WAHLSTER, W.; HELBIG, J. Recommendations for implementing the strategic initiative INDUSTRIE 4.0. Frankfurt: Acatech, 2013.
- LAUDON, K. C.; LAUDON, J. P. Sistemas de informação gerenciais. 11. ed. São Paulo: Pearson Education do Brasil, 2014.
- MARTIN, R. C. Clean architecture: a craftsman's guide to software structure and design. Boston: Prentice Hall, 2018.
- MCKINNEY, W. Python for data analysis. 3. ed. Sebastopol: O'Reilly Media, 2022.
- MONGODB INC. MongoDB documentation. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 2 maio 2026.
- O'BRIEN, J. A.; MARAKAS, G. M. Administração de sistemas de informação. 15. ed. Porto Alegre: AMGH, 2013.
- POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 1 maio 2026.
- PRESSMAN, R. S.; MAXIM, B. R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2020.
- PSYCOPG DEVELOPMENT TEAM. Psycopg documentation. Disponível em: <https://www.psycopg.org/docs/>. Acesso em: 1 maio 2026.
- SADALAGE, P. J.; FOWLER, M. NoSQL distilled: a brief guide to the emerging world of polyglot persistence. Boston: Addison-Wesley, 2013.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. Sistema de banco de dados. 7. ed. Rio de Janeiro: Elsevier, 2020.

SLACK, N.; BRANDON-JONES, A.; JOHNSTON, R. Administração da produção. 8. ed. São Paulo: Atlas, 2018.

SOMMERVILLE, I. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

STAIR, R.; REYNOLDS, G. Princípios de sistemas de informação. 11. ed. São Paulo: Cengage Learning, 2015